

Expo Camera & ImagePicker Implementation

This project demonstrates the implementation of **expo-camera** and **expo-image-picker** for capturing photos and selecting images from the device gallery.

Installation

Install the required packages:

```
npm install expo-camera expo-image-picker
```

Dependencies

- **expo-camera**: ^17.0.10
- **expo-image-picker**: ^17.0.10

Expo Camera

Features

- **Camera View**: Full-screen camera preview
- **Take Picture**: Capture photos with a single tap
- **Flip Camera**: Switch between front and back camera
- **Permission Handling**: Automatic permission requests

Implementation

The camera component is located at **components/camera.js**.

Key Components

```
import { CameraView, useCameraPermissions } from 'expo-camera';
```

Usage Example

```
import Camera from '../components/camera';

function MyScreen() {
  const handlePhotoTaken = (photoUri) => {
    console.log('Photo captured:', photoUri);
    // Handle the captured photo
  };
}
```

```
    return <Camera onPhotoTaken={handlePhotoTaken} />;  
  }
```

Camera Component Features

1. Permission Management

- Automatically requests camera permissions
- Shows permission request UI if not granted
- Handles permission states gracefully

2. Camera Controls

- **Flip Camera Button:** Toggle between front/back camera
- **Capture Button:** Circular button to take pictures
- **Take Picture Text:** Label below capture button

3. Photo Capture

- Uses `takePictureAsync()` method
- Returns photo URI via callback
- Handles errors gracefully

Code Structure

```
export default function Camera({ onPhotoTaken }) {  
  const [facing, setFacing] = useState('back');  
  const [permission, requestPermission] = useCameraPermissions();  
  const cameraRef = useRef(null);  
  
  async function takePicture() {  
    const photo = await cameraRef.current.takePictureAsync();  
    if (onPhotoTaken) {  
      onPhotoTaken(photo.uri);  
    }  
  }  
  
  return (  
    <CameraView  
      ref={cameraRef}  
      facing={facing}  
      style={styles.camera}  
    />  
  );  
}
```



Features

- **Gallery Access:** Select images from device photo library
- **Image Editing:** Crop and edit selected images
- **Permission Handling:** Request media library permissions
- **Quality Control:** Adjust image quality settings

Implementation

ImagePicker is used in `Screens/PostScreen.js`.

Usage Example

```
import * as ImagePicker from 'expo-image-picker';

const pickImage = async () => {
  // Request permissions
  const { status } = await ImagePicker.requestMediaLibraryPermissions();

  if (status !== 'granted') {
    Alert.alert('Permission needed', 'We need permission to access your photos!');
    return;
  }

  // Launch image picker
  const result = await ImagePicker.launchImageLibraryAsync({
    mediaTypes: ImagePicker.MediaTypeOptions.Images,
    allowsEditing: true,
    aspect: [4, 3],
    quality: 1,
  });

  if (!result.canceled) {
    setSelectedImage(result.assets[0].uri);
  }
};
```

Configuration Options

- `mediaTypes`: Type of media to pick (Images, Videos, or All)
- `allowsEditing`: Enable image cropping/editing
- `aspect`: Aspect ratio for editing (e.g., [4, 3])
- `quality`: Image quality (0-1, where 1 is highest)

Permissions

Camera Permissions

The app automatically handles camera permissions using `useCameraPermissions()`:

```
const [permission, requestPermission] = useCameraPermissions();

if (!permission.granted) {
  // Show permission request UI
  return <Button onPress={requestPermission} title="Grant Permission" />;
}
```

Media Library Permissions

Request media library permissions before accessing the gallery:

```
const { status } = await ImagePicker.requestMediaLibraryPermissionsAsync();
```

Usage in PostScreen

The **PostScreen** component demonstrates both features:

1. **Take Photo**: Opens camera component
2. **Upload Image**: Opens image picker from gallery
3. **Image Preview**: Shows selected/captured image
4. **Remove Image**: Option to remove selected image

Example Flow

```
// Take photo with camera
const handleCameraPhoto = (photoUri) => {
  setSelectedImage(photoUri);
  setShowCamera(false);
};

// Pick image from gallery
const pickImage = async () => {
  const result = await ImagePicker.launchImageLibraryAsync({
    mediaTypes: ImagePicker.MediaTypeOptions.Images,
    allowsEditing: true,
    aspect: [4, 3],
    quality: 1,
  });

  if (!result.canceled) {
    setSelectedImage(result.assets[0].uri);
  }
};
```

UI Components

Camera Component UI

- **Camera View:** Full-screen camera preview
- **Flip Button:** Icon button with "Flip Camera" text
- **Capture Button:** Large circular white button
- **Take Picture Label:** Text below capture button

Image Picker UI

- **Take Photo Button:** Opens camera
- **Upload Image Button:** Opens gallery picker
- **Image Preview:** Displays selected image
- **Remove Button:** X button to remove image



File Structure

```
expo-app-v4/  
├── components/  
│   └── camera.js           # Camera component  
├── Screens/  
│   └── PostScreen.js       # Post screen with camera & image picker  
└── README.md               # This file
```



Getting Started

1. Install dependencies:

```
npm install
```

2. Start the development server:

```
npm start
```

3. Run on device/emulator:

```
npm run android # For Android  
npm run ios     # For iOS
```



Important Notes

- **Permissions:** Both camera and media library require user permissions
- **Platform Support:** Works on iOS, Android, and Web (with limitations)
- **Image Quality:** Adjust quality settings based on your needs

- **Error Handling:** Always handle permission denials and errors gracefully

Resources

- [Expo Camera Documentation](#)
- [Expo ImagePicker Documentation](#)

Troubleshooting

Camera not working?

- Check if permissions are granted
- Verify camera is available on the device
- Check for proper ref usage in CameraView

Image Picker not working?

- Ensure media library permissions are granted
- Check if device has photos in gallery
- Verify ImagePicker configuration options

License

This project is part of a course assignment.